

Tuple in Python

A **Tuple** is used to store multiple values in one variable.

Tuples are:

- Ordered
 - Immutable (cannot be changed)
 - Allow duplicate values
-

Creating a Tuple

Code

```
numbers = (10, 20, 30)
```

```
print(numbers)
```

Output

```
(10, 20, 30)
```

Difference Between List and Tuple

List	Tuple
Uses []	Uses ()
Mutable	Immutable
Can change values	Cannot change values

Accessing Tuple Elements

Index starts from 0.

Code

```
numbers = (10, 20, 30, 40)
```

```
print(numbers[0])
```

```
print(numbers[2])
```

Which is What?

- numbers[0] → first element
 - numbers[2] → third element
-

Output

10

30

Negative Indexing

Code

```
numbers = (10, 20, 30, 40)
```

```
print(numbers[-1])
```

Understanding

- -1 means last element

Output

40

Tuple Slicing

Code

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[1:4])
```

Understanding

- Start = 1
 - Stop = 4
 - Stops before index 4
-

Output

(20, 30, 40)

Tuple is Immutable

You cannot change tuple elements.

Wrong Code

```
numbers = (10, 20, 30)
```

```
numbers[1] = 100
```

Output

```
TypeError
```

Tuple with One Element

Must use comma ,

Code

```
t = (5,)
```

```
print(t)
```

Output

```
(5,)
```

Without Comma

Code

```
t = (5)
```

```
print(type(t))
```

Output

```
<class 'int'>
```

Because Python treats it as integer.

Loop Through Tuple

Code

```
numbers = (10, 20, 30)
```

```
for n in numbers:
```

```
    print(n)
```

Output

```
10
```

```
20
```

```
30
```

Length of Tuple

Code

```
numbers = (10, 20, 30)
```

```
print(len(numbers))
```

Output

```
3
```

Membership Operators

Code

```
numbers = (10, 20, 30)
```

```
print(20 in numbers)
```

```
print(50 in numbers)
```

Output

```
True
```

```
False
```

Tuple Concatenation

Joining tuples.

Code

```
a = (1, 2)
```

```
b = (3, 4)
```

```
c = a + b
```

```
print(c)
```

Output

```
(1, 2, 3, 4)
```

Tuple Repetition**Code**

```
a = (1, 2)
```

```
print(a * 3)
```

Output

```
(1, 2, 1, 2, 1, 2)
```

Tuple Packing

Putting multiple values into tuple.

Code

```
t = 10, 20, 30
```

```
print(t)
```

Output

```
(10, 20, 30)
```

Tuple Unpacking

Extracting values from tuple.

Code

```
t = (10, 20, 30)
```

```
a, b, c = t
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

Output

```
10
```

```
20
```

```
30
```

Important Tuple Functions

1. count()

Counts occurrences.

Code

```
numbers = (10, 20, 10, 30)
```

```
print(numbers.count(10))
```

Output

```
2
```

2. index()

Finds position.

Code

```
numbers = (10, 20, 30)
```

```
print(numbers.index(20))
```

Output

```
1
```

Nested Tuple

Tuple inside another tuple.

Code

```
numbers = ((1, 2), (3, 4))
```

```
print(numbers[0])
```

```
print(numbers[1][0])
```

Output

```
(1, 2)
```

```
3
```

Convert Tuple to List

Code

```
t = (10, 20, 30)
```

```
l = list(t)
```

```
print(l)
```

Output

```
[10, 20, 30]
```

Convert List to Tuple

Code

```
l = [10, 20, 30]
```

```
t = tuple(l)
```

```
print(t)
```

Output

```
(10, 20, 30)
```

Advantages of Tuple

1. Faster than list

2. Data is safe (immutable)
3. Uses less memory

Important Tuple Operations

Operation Purpose

len()	Length
count()	Count values
index()	Find position
+	Concatenate
*	Repeat
in	Membership

Most Important for Exams

Usually asked:

1. Definition of tuple
2. Difference between list and tuple
3. Tuple packing/unpacking
4. count() and index()
5. Tuple slicing
6. Nested tuple
7. Immutable property

Quick Revision

Feature	Tuple
Symbol	()
Ordered	Yes
Mutable	No
Duplicates allowed	Yes
Indexed	Yes

